

# Information, Belief, and Search in MTGallium

MTGallium

Version 2.0 / 23 September 2026

## Abstract

MTGallium's policies play Magic without seeing the opponent's hand or the order of either library. This paper defines what a player knows, how a policy weighs the possibilities it cannot rule out, what a value estimate means, and what MTGallium's search computes. Each term it defines is the one used in the code and in [docs/terminology.md](#); [section 8](#) maps each one to its source. One small example runs throughout.

## 1 Why a model

A program that plays a hidden-information game can go wrong in ways that still produce reasonable-looking moves. It can peek at the referee's state through a side channel. It can average over guesses about the opponent's hand and then act as if it knew which guess was right. It can report a number from its search without saying what question that number answers. Each of these errors is easy to state precisely and hard to see in a game log. This paper gives the definitions needed to state them precisely.

[Section 2](#) describes the game and what each player observes. [Section 3](#) turns observations into probabilities, and [section 4](#) defines values. [Section 5](#) describes the search, and [section 6](#) states what keeps it honest. [Sections 7](#) and [8](#) connect the model to the implementation, and [section 9](#) states what the model leaves out.

**The running example: holding up Counterspell.** We call the player we are planning for *Rae* and her opponent *Oli*. Oli has one card in hand. It was drawn from a library holding one Counterspell and two lands, so it is Counterspell (C) with probability  $1/3$  and a land (L) otherwise. Oli sees the card; Rae does not.

1. Oli either *holds up* mana (*H*) or *taps out* (*T*). Rae sees which. If he taps out, nothing can be countered this turn, so Rae casts her threat and wins.
2. If Oli holds up mana, Rae either *casts* her threat now or casts a *bait* spell first. She has mana for one spell a turn, so baiting delays the threat by a turn.
3. Casting now wins (+1) if Oli holds a land and loses (−1) if he counters it.
4. After a bait, Oli with Counterspell must decide whether to spend it on the bait. He cannot tell whether the bait is Rae's best card or a decoy for a stronger follow-up, so assume he counters it half the time. If he does, or if he holds only a land, the threat resolves a turn late. If he does not, he counters the threat next turn (−1).

A threat that resolves a turn late has given Oli an extra draw to find an answer. Say Rae then wins three games in four, so her expected payoff is  $0.75(+1) + 0.25(-1) = +0.5$ . Payoffs are Rae's; Oli's are their negatives. Whether Rae should cast or bait after seeing  $H$  depends on how likely Oli is to hold Counterspell, and that depends on how Oli plays.

## 2 The game and what players observe

### 2.1 States, chance, and histories

**Definition 1** (Game). A two-player game with chance consists of

- a set of **states**  $S$  with an initial state  $s_0$  and terminal states  $F \subseteq S$ ;
- for each nonterminal state  $s$ , an **actor**  $p(s) \in \{1, 2, \mathbf{c}\}$ , where  $\mathbf{c}$  denotes **chance**, and a finite nonempty set of **legal actions**  $A(s)$ ;
- a deterministic **transition**  $s' = \tau(s, a)$  for each legal action;
- a fixed distribution  $\kappa(\cdot | s)$  over  $A(s)$  at each chance state;
- a **terminal payoff**  $u_i(s) \in [-1, 1]$  for each player at each terminal state, with  $u_1 = -u_2$ . A win is  $+1$  and a draw is  $0$ .

A state is the complete configuration, including every hidden card and the order of every library. A transition applies an action and all of its automatic consequences, and stops at the next point where a player or chance must choose. In Magic, a player choice is anything the rules ask a player to decide: casting, targeting, ordering triggers, declaring blockers, keeping or mulliganing. Chance covers every shuffle, die roll, coin flip, and random discard, as well as the opening deal, which is why a single initial state is enough. A shuffle is one chance action whose outcome is the new library order.

A **history**  $h = (s_0, a_0, s_1, \dots, a_{n-1}, s_n)$  is a sequence of legal steps. Its **reach probability** under a pair of policies  $\pi = (\pi_1, \pi_2)$  (section 3) is the product of the probabilities of its actions, where  $h_{\leq k}$  is the prefix of  $h$  ending at  $s_k$ :

$$\mathbb{P}^\pi(h) = \prod_{k < n} \begin{cases} \kappa(a_k | s_k), & p(s_k) = \mathbf{c}, \\ \pi_{p(s_k)}(a_k | x_{p(s_k)}(h_{\leq k})), & \text{otherwise.} \end{cases} \quad (1)$$

### 2.2 Observations and information states

**Definition 2** (Observations and information state). Each step  $(s, a, s')$  gives player  $i$  a possibly empty sequence of **observations**  $o_i(s, a, s')$ . The **information state** of player  $i$  after history  $h$  is everything that player has observed, in order:

$$x_i(h) = o_i(s_0, a_0, s_1) o_i(s_1, a_1, s_2) \cdots o_i(s_{n-1}, a_{n-1}, s_n).$$

We require two things of the observations, both true of Magic:

- (O1) *A player observes their own actions.* When  $p(s) = i$ , the sequence  $o_i(s, a, s')$  begins with  $a$ . In particular, each own action is recorded after everything the player had seen when choosing it.

(O2) *A player knows when they must act, and their options.* Whether  $p(s_n) = i$ , and if so  $A(s_n)$ , are determined by  $x_i(h)$ .

If two situations the player cannot tell apart offered different legal actions, the menu itself would reveal hidden information. Requirement (O2) says that such a difference must also appear as an observation.

Observations include public events (a spell is cast), private reveals (looking at the top card of a library), one's own draws, and the result of a player's own choices. An opponent's hidden action, such as choosing which card to discard face down, produces an observation that it happened but not which card was chosen.

**Definition 3** (Information set). The **information set** of an information state  $x$  for player  $i$  is the set of histories that player cannot rule out:  $\mathcal{H}_i(x) = \{h : x_i(h) = x\}$ . This is the usual extensive-form notion [5], indexed by the observation sequence.

In the example, after Oli holds up mana Rae's information state is the same whichever card Oli holds, so her information set contains two histories, one with C and one with L.

## 2.3 Worlds

A current state alone is not enough to continue a game. Two histories can reach the same cards in the same zones while the opponent remembers different things: in one, Rae's hand was revealed to Oli earlier; in the other, it was not. Oli's future play can depend on that memory.

**Definition 4** (World). The **world** of a history is its current state together with both players' information states:  $w(h) = (s_n, x_1(h), x_2(h))$ .

**Proposition 1** (A world is enough to continue). *If each player's policy depends only on their own information state, then two histories with the same world have the same distribution of future play and of terminal payoff.*

*Proof.* The state determines the actor, legal actions, chance distribution, transition, observations, and payoff. The actor's information state determines the policy's action distribution. After each action both parts of the world are updated from the same inputs, so the argument repeats until the game ends.  $\square$

A hypothetical continuation must therefore be run from a world, not from a state. MTGallium's search runs simulations on objects called `SearchWorlds` for this reason. Each one carries an engine state together with each player's represented history and knowledge.

## 3 Beliefs

### 3.1 Policies

A **policy** for player  $i$  assigns each information state at which  $i$  must act a probability distribution  $\pi_i(\cdot | x)$  over the legal actions. A policy can see only its own player's information state. The policy we assume the opponent follows is the **opponent model**, written  $\mu$ .

### 3.2 Belief over a player's information set

**Definition 5** (Belief). A **belief** of player  $i$  at information state  $x$  is a probability distribution  $b$  over the information set  $\mathcal{H}_i(x)$ . Its **support** is the set of histories with positive probability.

Mapping each history to its world turns a belief over histories into a belief over worlds, and search samples worlds. The mapping keeps the pairing between hidden cards and what the opponent remembers. Sampling the state and the opponent's memory independently would produce worlds that no history could have reached.

A belief is different from **knowledge**. A fact is known at  $x$  when it holds in every history of  $\mathcal{H}_i(x)$ , such as the identity of a card the player saw placed on top of a library and not moved since. Knowledge excludes only histories that are logically incompatible with what the player observed. A belief may also assign zero probability to a compatible history, for example when the opponent model gives an observed action probability zero. That exclusion rests on a modeling assumption; it is not knowledge. MTGallium tracks known facts separately from beliefs and uses them to reject hypothetical worlds that contradict them.

### 3.3 The posterior, and why only opponent actions carry evidence

Suppose the past was generated by chance, by player  $i$ 's own policy, and by the opponent model  $\mu$ , and that under these assumptions  $x$  had positive probability. Conditioning [equation \(1\)](#) on the information state gives the posterior  $b(h) \propto \mathbb{P}(h)$  over  $h \in \mathcal{H}_i(x)$ . Split each reach probability into its chance, own, and opponent factors. By (O1), every history in the information set contains the same sequence of the player's own choices, each made after the same observations, so every own-choice probability is the same across those histories. The own factor is therefore a common positive constant, and it cancels, leaving

$$b(h) \propto \underbrace{\prod_{\text{chance steps}} \kappa(a_k | s_k)}_{\text{chance}} \underbrace{\prod_{\text{opponent steps}} \mu(a_k | x_{-i}(h_{\leq k}))}_{\text{opponent behavior}}, \quad h \in \mathcal{H}_i(x). \quad (2)$$

A player learns nothing from their own choices beyond what those choices caused to be observed. Opponent choices are evidence, because the opponent chose them with information the player lacks.

MTGallium has two ways of constructing beliefs (`BeliefMode`):

- A **consistency belief** keeps only the chance factor. It deals the unseen cards uniformly from what remains of the known decklists, and rejects deals that contradict represented knowledge. It ignores what the opponent's choices suggest about their hand.
- A **policy-conditioned belief** is also conditioned on the opponent's policy: it includes the opponent factor, reweighting worlds by how likely the opponent model was to make each observed choice.

**Example 1** (Reading Oli's mana). Let the opponent model be  $\mu(H | C) = 0.9$  and  $\mu(H | L) = 0.3$ . After Oli holds up mana,

$$b(C) = \frac{\frac{1}{3} \cdot 0.9}{\frac{1}{3} \cdot 0.9 + \frac{2}{3} \cdot 0.3} = 0.6.$$

The consistency belief keeps the prior  $b(C) = 1/3$ . After Oli taps out, the policy-conditioned belief gives  $b(C) = 1/15$ .

### 3.4 Particles

Exact beliefs over Magic hands are too large to enumerate. MTGallium represents a belief as a set of **particles**: complete weighted worlds. When the real game advances, a policy-conditioned belief updates each particle according to who acted:

- *The player's own action, or chance.* Advance the world through the step. The weight is unchanged, since own actions carry no evidence (equation (2)).
- *An observed opponent action.* Advance the world through that action and multiply the weight by the opponent model's probability of it in that world. This is the opponent factor of equation (2).
- *A hidden opponent action*, such as a face-down discard. Draw the action from the opponent model, using the opponent's information in that world, and advance. The weight is unchanged, because drawing the action already accounts for its probability; multiplying by it as well would count the same evidence twice.

In every case, then discard worlds whose new information state for the player disagrees with what the player actually observed, normalize the weights, and resample.

## 4 Values

### 4.1 A value answers a stated question

Let  $G_i^\pi(h)$  be player  $i$ 's expected terminal payoff when play continues from  $h$  under policies  $\pi$  and chance. For a belief  $b$  at an information state where  $i$  must act,

$$V_i(b, \pi) = \sum_h b(h) G_i^\pi(h), \quad (3)$$

$$Q_i(b, a, \pi) = \sum_h b(h) G_i^\pi(h \text{ followed by } a). \quad (4)$$

$V$  is the **value** of the position and  $Q$  is the **action value**: player  $i$  takes  $a$  now and follows  $\pi$  afterwards, even if  $\pi$  itself would never choose  $a$ .

A value therefore has three inputs: *whose payoff*, *which belief*, and *which continuation policies*. We call this triple the value's **target**. Two numbers with different targets are answers to different questions, and comparing them says nothing about which answer is more accurate.

**Example 2** (Cast or bait). After Oli holds up mana, baiting is worth  $\frac{1}{2}(+0.5) + \frac{1}{2}(-1) = -0.25$  when he has C, and  $+0.5$  when he has L.

| Belief                                  | $Q(\text{cast})$                           | $Q(\text{bait})$                               |
|-----------------------------------------|--------------------------------------------|------------------------------------------------|
| Consistency, $b(\text{C}) = 1/3$        | $\frac{1}{3}(-1) + \frac{2}{3}(+1) = 0.33$ | $\frac{1}{3}(-0.25) + \frac{2}{3}(0.5) = 0.25$ |
| Policy-conditioned, $b(\text{C}) = 0.6$ | $0.6(-1) + 0.4(+1) = -0.20$                | $0.6(-0.25) + 0.4(0.5) = 0.05$                 |

The two beliefs recommend different actions. Neither is miscalculated; they rest on different assumptions about Oli.

## 4.2 Evaluators

Search usually has to stop before the game ends and score the position it reached. There are two kinds of scorer:

- An **information-state evaluator**  $e_i(x)$  scores one player's information state. It cannot use information unavailable to that player, so every world sharing that information state gets the same score.
- A **world evaluator**  $g_i(w)$  scores a complete world, including the opponent's hidden cards.

An evaluator's output is only a value estimate once its target is stated, usually implicitly through the games it was trained or tuned on, and that estimate still has to be validated. Putting a score in  $[-1, 1]$  does not make it an expected terminal payoff.

Seeing the whole world is not itself a mistake. By [proposition 1](#), the expected payoff  $G_i^\pi$  of a fixed policy pair depends only on the world, so a world evaluator that predicts it is legitimate, and averaging its predictions over a belief gives exactly [equation \(3\)](#). A world evaluator goes wrong when its score assumes the player will choose differently in worlds she cannot tell apart, which is what a score learned from perfect-information play, or from solving each world, implicitly does. The next section shows how much that assumption is worth.

## 5 Search

### 5.1 Why clairvoyant choices overvalue a position

A **determinization** is a complete world sampled from a belief. The simplest way to plan with hidden information is to sample determinizations, solve each as if nothing were hidden, and average the results. Averaging the best result of each world gives at least the value of the best single decision:

$$\sum_w b(w) \max_a Q(w, a) \geq \max_a \sum_w b(w) Q(w, a). \quad (5)$$

The left side assumes the player picks the best action for each world separately. That player cannot, because every world in the belief looks the same to her. The literature calls this error *strategy fusion* [\[2, 4\]](#).

**Example 3** (The value of clairvoyance). Under the policy-conditioned belief, Rae's best action against C is to bait (worth  $-0.25$ ) and against L is to cast (worth  $+1$ ). Averaging those gives  $0.6(-0.25) + 0.4(+1) = 0.25$ . Rae's actual best is to bait, worth  $0.05$ . The difference,  $0.20$ , is what knowing Oli's card would be worth.

### 5.2 The search tree contains only the root player's decisions

MTGallium's search builds its tree from the decisions of the player it is choosing for, called the **root player**. Each tree **node** is one root-player information state, together with the menu of actions search considers there. Different sampled worlds that give the root player the same information state reach the same node, so the root player makes the same choice in all of them, just as in the real game. This rules out strategy fusion within the tree. It does not stop a leaf evaluator from supplying clairvoyant values ([section 4](#)).

Every other step happens inside the sampled world and adds no tree node. Chance draws from the world’s own chance stream ([section 6.1](#)). The opponent chooses with the opponent model  $\mu$ , applied to the opponent’s information state in that world. In the example, Rae’s search has a single node after  $H$ , with the edges *cast* and *bait* ([figure 1](#)). Oli’s card and his response to the bait are drawn inside each simulation.

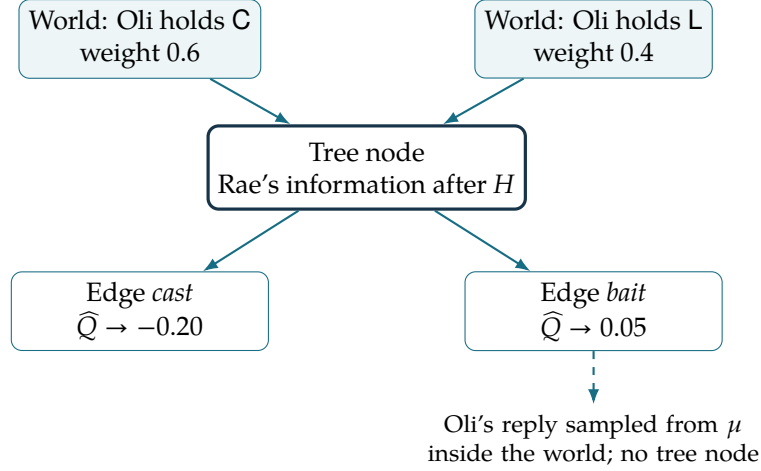


Figure 1: Rae’s search in the example. Both sampled worlds reach one tree node, because Rae cannot tell them apart. With enough simulations, each edge’s mean approaches its action value under the policy-conditioned belief.

One **simulation** proceeds as follows:

1. Sample a particle from the root belief and copy its world.
2. At each root-player node  $n$ , try untried actions first, and otherwise choose the action maximizing the UCT (upper confidence bounds applied to trees) score [3]

$$\widehat{Q}(n, a) + c \sqrt{\ln(N(n) + 1) / N(n, a)},$$

where  $N$  counts earlier simulations through the node or edge,  $\widehat{Q}$  is their mean result, and  $c$  is the exploration coefficient: larger values try less-visited actions more often.

3. Continue until the game ends or the simulation takes an edge for the first time. At that new **leaf**, obtain a **settlement**: one number in  $[-1, 1]$  from the root player’s perspective ([section 5.4](#)).
4. **Back up** the settlement: add it to every edge the simulation took.

After the budget is spent, search plays the root action with the most visits, breaking ties by higher mean and then by a fixed ordering of actions.

### 5.3 Why the opponent is a model, not a minimizer

An alternative would give the opponent tree nodes too, choosing whatever minimizes the root player’s results so far. Inside the root player’s search, however, every sampled world contains the root player’s actual hand. An opponent that learns from those results learns to answer that hand, which it cannot see: it would block exactly when the root player really

holds the combat trick. Cowling, Powley, and Whitehouse discuss this form of leakage in information-set search [1, Section X].

Choosing opponent moves with a fixed model  $\mu$  that sees only the opponent's information avoids this leakage. Once  $\mu$  is fixed, the root player faces a single-agent problem with hidden state, a partially observable Markov decision process (POMDP), as in [6]. The search then seeks a *best response to  $\mu$* : the root player's best play assuming the opponent really plays like  $\mu$ . It makes no equilibrium claim. Search is only as strong as the opponent model is realistic, and an opponent who plays unlike  $\mu$  can exploit it.

## 5.4 Leaves and settlements

A settlement is produced in one of four ways, and search records which one, its **settlement origin** (`ReturnSource`):

- *Terminal payoff*. The game ended in the simulation.
- *Heuristic settlement*. An evaluator scored the leaf. This is either an information-state evaluator on the root player's information state or a world evaluator on the sampled world.
- *Learned outcome estimate*. A learned evaluator that declares itself an estimate of terminal payoff scored the leaf.
- *Neutral settlement*. The position could not be settled within its limits and was configured to count as 0. This scores it as a draw.

The position scored can be the leaf itself, the end of a **rollout** (continued play under separate rollout policies for each seat, for a bounded number of decisions or turns), or a quiet position reached from either. A **quiet** position has an empty stack, no combat in progress, no pending combat-damage or ordering decision, and no creature with lethal damage marked. To reach one, search takes only **forced** priority passes, where passing is the only legal action. Two optional variants go further: one also takes a pass that is the only action the configured action space admits while the position is not yet quiet, and another lets the rollout policies make the remaining choices. "Quiet" is this checklist, not a claim that the tactical consequences of the position have played out.

The root player's information-state evaluator is used even when the opponent is next to act. It cannot use the opponent's sampled hidden cards, so the sampled world affects its score only through the moves and chance outcomes that the world produced along the way.

If a sampled world refuses an action its own menu offered, the whole search fails; the refusal is not given any value. Skipping the refused simulation and continuing would average only over worlds that happen to execute, which biases the result in a way nobody asked for.

## 5.5 What the numbers in a search result mean

- An edge's **visits** count the settlements added to it. Its **search mean**  $\widehat{Q}$  is their average.
- The visits to an edge follow the root player's later choices as the search currently makes them, and those choices change as it learns. Their settlements may mix terminal payoffs, evaluator scores, and imposed zeros. The search mean therefore need not estimate the terminal payoff of the action finally chosen, and a visit count does not by itself establish statistical confidence.

- At an edge below the root, the worlds that arrive follow the root belief updated by the moves and chance outcomes on the way there, not the root belief itself.
- The **root value** is the visit-weighted mean over all root edges. It describes the mixture of actions the search tried, not the chosen action.
- The settlement counts per edge report how many of its visits came from terminal payoffs, and how many from each kind of estimate.
- Stopping at a wall-clock budget makes the number of simulations depend on how long each one took. Report the budget that was actually used.

## 6 Keeping the root honest

Search looks at hidden cards all the time: every simulation runs on a complete world. This is legitimate as long as those worlds were generated from what the root player knows. The requirement is about where the inputs came from.

**Perspective safety** is the design rule that the distribution of the chosen action depends on the actual game only through the root player's information state  $x$ . It holds when the particles, the menus, the chance streams, and all random seeds are computed from  $x$  and from randomness independent of the actual game, and every other step reads only these inputs and what the search derived from them. To see why, run the search for two actual histories with the same  $x$  and the same seeds: every input is equal, so every step computes the same values and makes the same choice.

Perspective safety is an information boundary. It says nothing about whether the beliefs are accurate or the values calibrated; those are the questions of [sections 3](#) and [4](#). The practical work is in closing every channel through which the actual game could leak:

- *Particles* are built from the root player's represented information and the known decklists, never from the referee's state. Search checks that every particle presents the same root information state and menu before it starts.
- *Menus* are functions of the acting player's information state (O2). A menu that differed between worlds the player cannot tell apart would leak hidden information.
- *Chance streams* are seeded from search randomness ([section 6.1](#)).
- *Identifiers* such as engine object ids, hashes, and action order are information too. Representations passed to policies use viewer-local references instead of raw engine ids.

### 6.1 Chance inside a hypothetical world

The engine resolves chance with a pseudorandom generator stored in the state. A hypothetical world must not reuse the referee's generator. That generator determines the actual game's future shuffles, so a simulation that used it would draw the root player's real future cards. MTGallium gives each hypothetical world a fresh **chance stream** seeded from the search's own randomness.

For a single continuation, fixing a world's future chance outcomes in advance changes nothing: an independent seed that only chance steps read produces the same distribution of play as drawing each outcome when it occurs. Because the seed is part of the world, and the tree

is keyed by the root player's information state, the seed never becomes something the root player can condition on.

Repeated continuations are another matter. Within one search, every simulation that draws the same particle replays its chance stream, so repeated simulations provide fewer independent chance samples than their visit count suggests. (When a policy-conditioned belief resamples, the copies it creates do get fresh chance streams, so duplicate particles do not share a future.) For a fixed continuation policy this adds variance without changing the expectation over independently drawn seeds. That qualification matters: for actions chosen adaptively, and for the search means they produce, there is no such general guarantee, and extra visits cannot remove the uncertainty of a fixed, finite set of sampled futures. Experiments can also schedule chance streams explicitly, to compare two configurations on the same random futures.

## 7 Representations

The program never holds the information state  $x$  itself. It holds a **representation**  $r(x)$ : in MTGallium, the current observation snapshot, the player-visible event history, the represented knowledge, and, at a decision, the admitted menu. Three distinct properties matter:

- **Safe:**  $r$  is computed from the information state alone. Equal information states give equal representations.
- **Complete:**  $r$  loses nothing. Different information states give different representations.
- **Sufficient** for a task:  $r$  keeps everything that task needs. For example, it may determine the legal menu, or the value under a particular target.

A representation containing only the current board is safe but incomplete. If Oli earlier revealed a Lightning Bolt from his hand in one history and a Mountain in another, a board-only representation gives both histories the same representation, and any evaluator using it must score them alike. No learning method can recover a distinction the input has erased. A complete representation is sufficient for any task determined by the information state, and an incomplete one can still be sufficient for a task that does not need what was dropped.

Representations are compared by digest. Tree nodes are keyed by the digest of the root player's representation together with its admitted menu, so the tree distinguishes exactly what the representation distinguishes.

**Actions.** Search chooses among **semantic choices**: descriptions of an action in terms the player can observe, such as "attack with the 2/2 that entered this turn". Before the engine applies a semantic choice, it is **rebound** to the corresponding native action in that world. An action is **legal** if the rules allow it, **proposed** if action generation produced it, **admitted** if it is on the menu search or the policy considers, and **accepted** once the engine applied it. A result should state which legal actions were not admitted.

## 8 Where each concept lives in the code

Links point to the public repository.

| Concept                          | Source                                                                                                                                                           |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Observation snapshot             | <a href="#">PlayerObservationSnapshot</a> , projected by <a href="#">PlayerObservationProjector</a>                                                              |
| Player-visible history           | <a href="#">InformationStateRecorder</a> , <a href="#">EventObservationProjector</a>                                                                             |
| Knowledge                        | <a href="#">PlayerKnowledge</a>                                                                                                                                  |
| Information-state representation | <a href="#">InformationStateRepresentation</a> ; without a menu, <a href="#">InformationState</a> ; with the acting player’s menu, <a href="#">DecisionPoint</a> |
| World                            | <a href="#">SearchWorld</a> , implemented by <a href="#">ArgentumSearchWorld</a>                                                                                 |
| Chance stream                    | <a href="#">futureChanceStreamIdentity</a> in <a href="#">ArgentumSearchWorld</a> ; <a href="#">SimulationWorldSchedule</a>                                      |
| Policy; opponent model           | <a href="#">ActionSelector</a> , <a href="#">ActionDistributionModel</a> , <a href="#">OpponentPolicy</a>                                                        |
| Belief, particles                | <a href="#">BeliefMode</a> ( <a href="#">UNIFORM_CONSISTENT_DEALS</a> , <a href="#">OPPONENT_MODEL_POSTERIOR</a> ); <a href="#">ParticleBelief</a>               |
| Evaluators                       | <a href="#">InformationStateEvaluator</a> ; <a href="#">LeafValueSource.Information</a>                                                                          |
| Search, nodes, UCT               | <a href="#">InformationSetSearch</a> ; node key <a href="#">TreeNodeKey</a>                                                                                      |
| Settlement, origin               | <a href="#">SimulationReturn</a> , <a href="#">ReturnSource</a> , <a href="#">ReturnSourceCounts</a>                                                             |
| Leaf and rollout settings        | <a href="#">LeafEvaluationConfig</a> , <a href="#">RolloutTurnHorizon</a>                                                                                        |
| Search result                    | <a href="#">InformationSetSearchResult</a>                                                                                                                       |
| Semantic choice, rebinding       | <a href="#">SemanticChoice</a> ; see the <a href="#">information and decision contracts</a>                                                                      |

## 9 What the model leaves out

- *Equilibrium*. Search seeks a best response to a fixed opponent model. It does not look for strategies that are hard to exploit.
- *Exactness of beliefs*. Particles approximate a belief, the consistency belief ignores opponent behavior, and knowledge tracking covers only the events the adapter represents exactly. The adapter reports tracking it cannot keep exact rather than guessing.
- *More than two players*. The definitions assume two players and a zero-sum payoff.
- *Reuse across decisions*. Every search starts from an empty tree. Reusing statistics under a new belief would require reweighting, which this paper does not define.

## References

- [1] Peter I. Cowling, Edward J. Powley, and Daniel Whitehouse. “Information Set Monte Carlo Tree Search.” *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2):120–143, 2012. doi:10.1109/TCAIG.2012.2200894.
- [2] Ian Frank and David Basin. “Search in games with incomplete information: a case study using Bridge card play.” *Artificial Intelligence*, 100(1–2):87–123, 1998. doi:10.1016/S0004-3702(97)00082-9.
- [3] Levente Kocsis and Csaba Szepesvári. “Bandit based Monte-Carlo Planning.” In *Machine Learning: ECML 2006*, LNCS 4212, pp. 282–293, 2006.

- [4] Jeffrey Long, Nathan R. Sturtevant, Michael Buro, and Timothy Furtak. “Understanding the Success of Perfect Information Monte Carlo Sampling in Game Tree Search.” In *Proceedings of the AAAI Conference on Artificial Intelligence*, 24(1):134–140, 2010. [doi:10.1609/aaai.v24i1.7562](https://doi.org/10.1609/aaai.v24i1.7562).
- [5] Martin J. Osborne and Ariel Rubinstein. *A Course in Game Theory*. MIT Press, 1994.
- [6] David Silver and Joel Veness. “Monte-Carlo Planning in Large POMDPs.” In *Advances in Neural Information Processing Systems 23*, 2010.